

Week 3 Lab (Task 2) - Bivariate Analysis: Continuous vs Continuous

Continuous vs Continuous variable

In this notebook we explore how continuous variables together. In doing so we will explore bivariate and basic multivariate analysis through,

- Scatter plots
- Bubble charts
- Correlation matrix & heatmap
- Line charts

Import Neccessary Libraries

```
import pandas as pd
from google.colab import drive
import plotly.express as px
import plotly.graph_objs as go
import numpy as np
```

```
drive.mount('/content/drive', force_remount=True)
data_folder = "drive/MyDrive/data"
```

Mounted at /content/drive

We've seen a multitude of ways in which we can analyse a categorical variable with a continuous variable. In this section, we detail how you can analyse continous variables together. For the first few examples we will look at some health data in the year 2018.

```
health = pd.read_csv(f"{data_folder}/health.csv")
health.head()
```

	GDP/Capita	MeanSchooling	InfantMortality	Life_exp	Country	Year	Country Type	
0	61598.536704	12.6	3.3	80.953659	Denmark	2018	Developed	
1	4134.987198	10.2	14.2	69.689000	Mongolia	2018	Developing	
2	82818.108162	13.4	3.6	83.753659	Switzerland	2018	Developed	
3	11633.498009	10.6	8.7	76.520000	Argentina	2018	Developing	
4	5805.675616	8.4	17.2	74.131000	Paraguay	2018	Developing	

Next steps: [Generate code with health](#) [New interactive sheet](#)

Start coding or [generate](#) with AI.

Scatterplot

What is scatterplot?

A scatterplot is used to get an insight on the relationship between two variables X and Y . The (x, y) coordinates are represented on a cartesian plane, and the plot itself can reveal patterns or correlations in the data, such as linear relationships or clusters of points.

Advantages

- Easy to interpret
- It is a useful tool to identify potential univariate and bivariate outliers
- Can also be used to get an insight as to whether two variables have positive or negative correlation (and the extent to which this is the case).
- Can be used as a tool alongside the [Variance Inflation Factor](#) to detect [multicollinearity](#) in [multiple linear regression models](#)
- We can make our analysis multivariate in nature by colouring and sizing the points depending on a third and fourth variable etc.

Disadvantages

- With large datasets, too many points can make the graph look messy

- Outliers (particularly y outliers) can distort the relationship between the two variables
- Gives us an insight on correlation, but is not sufficient for determining causality

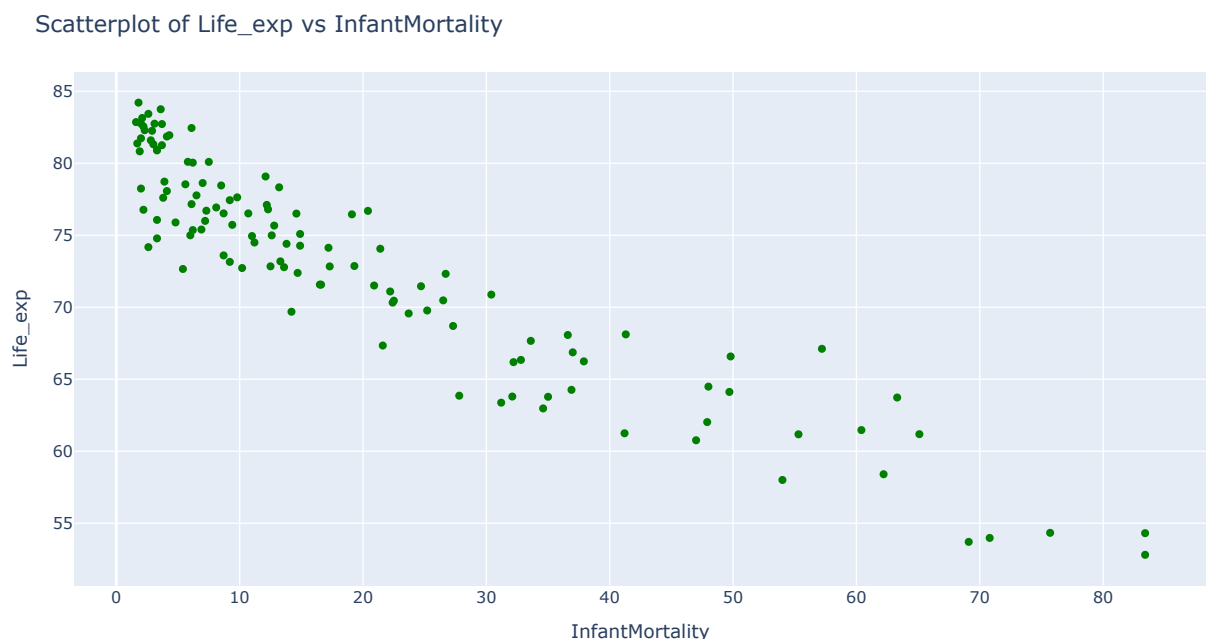
The Task

Here we take a look at the relationship between infant mortality and life expectancy across a variety of different countries in the year 2018.

Scatterplot: Plotly Express

With plotly express, we simply call the `scatter` top-level function and pass in the `x` and `y` columns as strings, assuming the `data_frame` argument references the correct data

```
fig = px.scatter(data_frame=health, x="InfantMortality", y="Life_exp", color_discrete_sequence=['green'])
fig.update_layout(title='Scatterplot of Life_exp vs InfantMortality')
fig.show()
```



Bubble Chart

What is Bubblechart?

A bubble chart is a scatter plot in which the size of the scatters is determined by a third variable

Advantages

- Can represent 3 variables on a two dimensional object
- Easy to see the relationship between the three variables
- Provides a more comprehensive view of the data since we have more variables we can extract insights from

Disadvantages

- Big bubbles can smother smaller ones, hiding them from the graph
- Requires adequate data cleaning of the third variable (controlling size) so that the graph does not become distorted

Your Turn

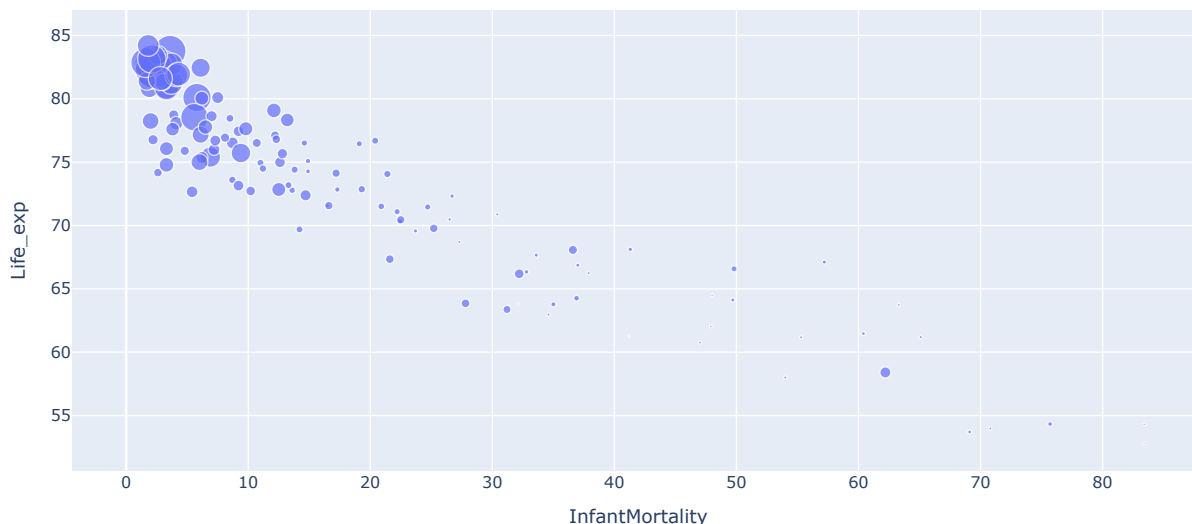
Look up the plotly docs for a Bubble chart and create a plot of life expectancy against infant mortality, denoting the size of the point through the corresponding value for GDP per capita.

```
# Create a bubble chart using Plotly Express
# x-axis: InfantMortality
```

```
# y-axis: Life_exp
# size of bubbles: GDP/Capita
fig = px.scatter(data_frame=health,
                 x="InfantMortality",
                 y="Life_exp",
                 size="GDP/Capita",
                 hover_name="Country", # Show country name on hover
                 title="Life Expectancy vs Infant Mortality (Bubble size: GDP/Capita)")

# Display the plot
fig.show()
```

Life Expectancy vs Infant Mortality (Bubble size: GDP/Capita)



Correlation Matrix & Heatmap

The Pearson's correlation coefficient (also known as Pearson's r) is a statistic that measures the degree to which two variables move together in linear fashion. Correlation takes on a value between -1 and 1, in which $r = -1$ implies perfect negative correlation (points move together in a perfect straight line with a negative gradient) and $r = 1$ implies perfect positive correlation (points move together in a perfect straight line with a positive gradient)

```
# We only have 2018 data hence we can't calculate correlation to do with year
# Drop the year column since we don't need it
corr = health.drop(columns='Year').corr(numeric_only=True)
corr
```

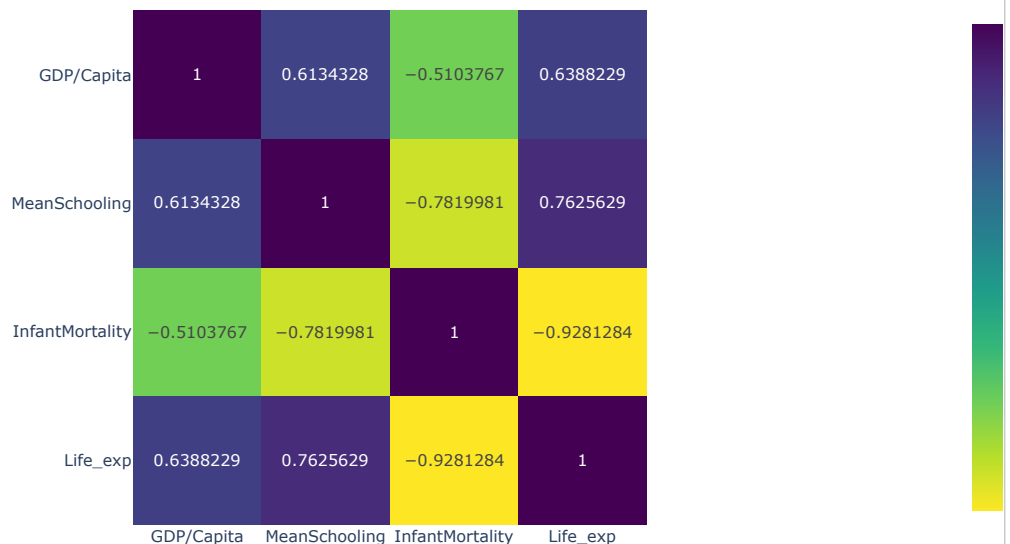
	GDP/Capita	MeanSchooling	InfantMortality	Life_exp	
GDP/Capita	1.000000	0.613433	-0.510377	0.638823	
MeanSchooling	0.613433	1.000000	-0.781998	0.762563	
InfantMortality	-0.510377	-0.781998	1.000000	-0.928128	
Life_exp	0.638823	0.762563	-0.928128	1.000000	

Next steps: [Generate code with corr](#) [New interactive sheet](#)

Correlation Matrix: Plotly Express

Here we use the `px.imshow()` top level function to plot our correlation matrix. We pass in our correlation matrix `corr` as the first argument and set `text_auto=True` to show the computed values on the plot. The `color_continuous_scale` argument provides us with a colour bar for colour coding purposes, though we will explore colour in more depth in the next chapter.

```
# Plot our correlation heatmap
fig = px.imshow(corr, text_auto=True, color_continuous_scale='viridis_r')
fig.show()
```



Line Charts and Multi Line Charts

What is Line chart?

Line charts are particularly useful visualisation options for when we wish to visualise patterns and trends of a continuous variable over *time*

Advantages

- Easy to visualise trends and patterns in data to see how a continuous variable changes over time
- Can plot many lines on a single chart
- Easy to identify areas of high fluctuation in the values of the data

Disadvantages

- Struggles to capture the way the data is distributed
- Can become messy as we add too many variables within a multi-line chart
- The plot can look extremely distorted even if a single value is incorrect or is an outlier

The task

In the following examples, we look at some stock data of the adjusted closing prices of popular companies. In a final example, we revisit more health data.

```
stocks = pd.read_csv(f'{data_folder}/stock_data.csv', parse_dates=['Date'])
stocks
```

	Date	AAPL	AMZN	GOOG	MSFT
0	2010-01-04	6.515214	6.695000	15.610239	23.683069
1	2010-01-05	6.526479	6.734500	15.541497	23.690718
2	2010-01-06	6.422663	6.612500	15.149715	23.545340
3	2010-01-07	6.410791	6.500000	14.797037	23.300472
4	2010-01-08	6.453411	6.676000	14.994298	23.461161
...	...	...	...	...	...
3267	2022-12-23	131.860001	85.250000	89.809998	238.729996
3268	2022-12-27	130.029999	83.040001	87.930000	236.960007
3269	2022-12-28	126.040001	81.820000	86.459999	234.529999
3270	2022-12-29	129.610001	84.180000	88.949997	241.009995
3271	2022-12-30	129.929993	84.000000	88.730003	239.820007

3272 rows × 5 columns

Next steps:

[Generate code with stocks](#)[New interactive sheet](#)

Line Chart: Plotly Express

To create a line chart in Plotly Express, we use the `px.line` top level function, passing in a `data_frame` as the first argument. Here, `x` represents your date column and `y` represents the variable you are interested in seeing its change over time.

```
fig = px.line(stocks, x='Date', y="AAPL", color_discrete_sequence=['black'])
fig.show()
```

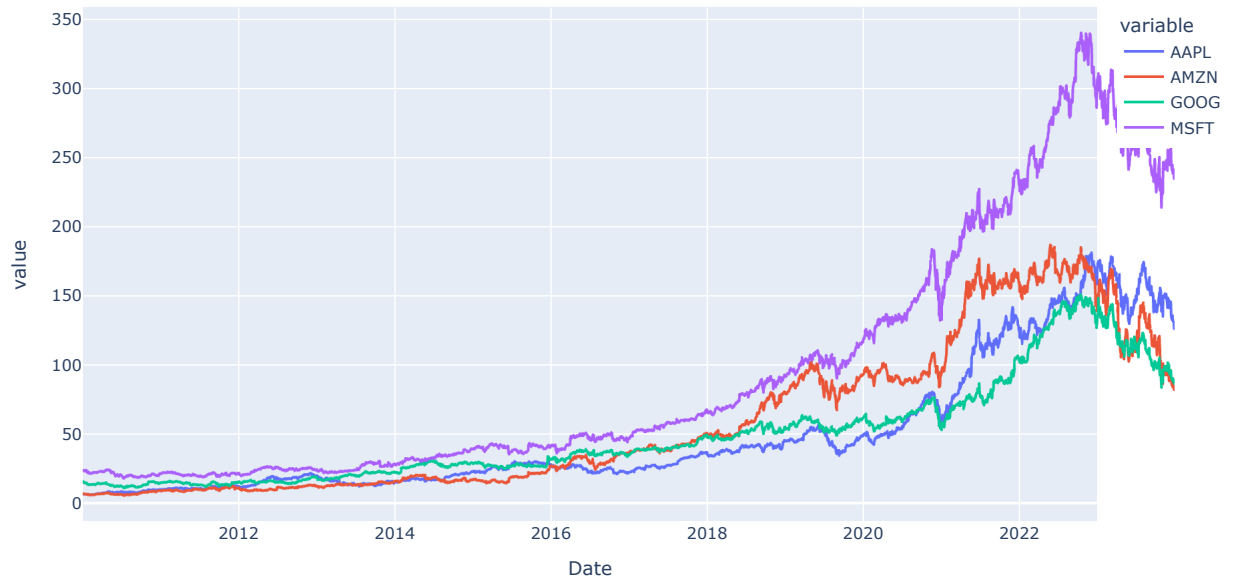


Multi-line Chart.

We have a variety of stocks in our dataset, including Apple, Amazon, Google and Microsoft. What if we want to include all of them in our dataset? All we have to do is pass an array-like object with **all** of the columns we wish to place on the line chart. As a shortcut, we have passed in all columns through `stocks.columns`, though if you wanted only Apple and Google, you could pass in a list.

Create a multi-line chart with a subset of stocks.

```
fig = px.line(data_frame=stocks, x='Date', y=stocks.columns)
fig.show()
```



▼ Your Turn

- Select at least 5-10 countries from the health dataset and create a multiline chart to compare development indicators.
- Hint: Normalise the numeric columns
- Add appropriate title, axis labels and legend

```
# Select 5-10 countries for comparison
selected_countries = [
    'Denmark', 'Mongolia', 'Switzerland', 'Argentina', 'Paraguay',
    'United States', 'Ethiopia', 'Japan', 'Norway', 'India'
]

# Filter the health data for selected countries
country_health = health[health['Country'].isin(selected_countries)].copy()

# Define numeric columns to normalize
numeric_cols = ['GDP/Capita', 'MeanSchooling', 'InfantMortality', 'Life_exp']

# Normalize the numeric columns using min-max scaling
for col in numeric_cols:
    min_val = country_health[col].min()
    max_val = country_health[col].max()
    country_health[f'{col}_Normalized'] = (country_health[col] - min_val) / (max_val - min_val)

# Melt the DataFrame for plotting multiple indicators
melted_health = country_health.melt(
    id_vars=['Country', 'Year'],
    value_vars=[f'{col}_Normalized' for col in numeric_cols],
    var_name='Indicator',
    value_name='Normalized Value'
)

# Create the multiline chart
fig = px.line(
    melted_health,
    x='Year',
    y='Normalized Value',
    color='Country',
    line_dash='Indicator',
    title='Normalized Development Indicators Across Selected Countries',
    labels={
        'Year': 'Year',
        'Normalized Value': 'Normalized Value (Min-Max Scaled)',
        'Country': 'Country',
        'Indicator': 'Indicator'
    }
)

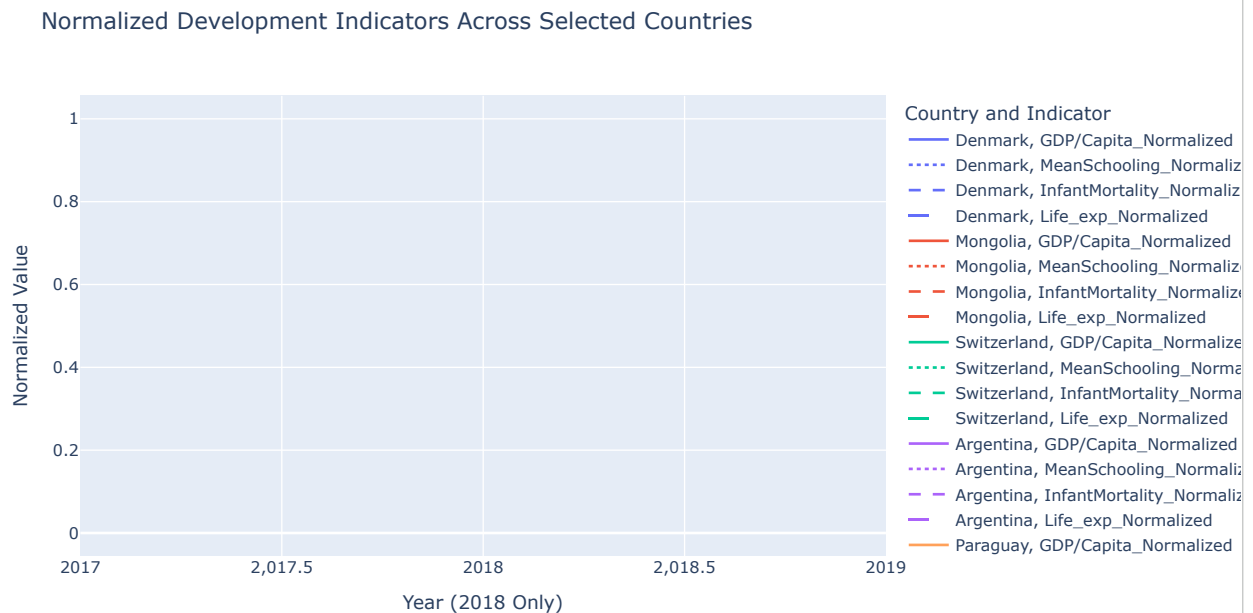
fig.update_layout(
    hovermode='x unified',
```

```

xaxis_title="Year (2018 Only)",
yaxis_title="Normalized Value",
legend_title="Country and Indicator"
)

fig.show()

```



Now, answer the following questions.

- Why is normalisation important before plotting these indicators together?
- What patterns do you observe between GDP, schooling, and life expectancy?

Try adding a different group of 5 countries and compare the results. What do you observe?

✓ Answers to the Questions

- **Why is normalisation important before plotting these indicators together?** Normalization (specifically min-max scaling, as used here) is crucial because the indicators (`GDP/Capita`, `MeanSchooling`, `InfantMortality`, `Life_exp`) have vastly different scales. For example, `GDP/Capita` can be in the tens of thousands, while `MeanSchooling` is typically a single or double-digit number. If plotted directly without normalization, the indicator with the largest scale (e.g., `GDP/Capita`) would dominate the chart, making it impossible to visually compare the trends and patterns of the other indicators. Normalization scales all values to a common range (e.g., 0 to 1), allowing for a fair and meaningful visual comparison of their relative changes and relationships.
- **What patterns do you observe between GDP, schooling, and life expectancy?** From the normalized multiline chart, we can generally observe a positive correlation between `GDP/Capita`, `MeanSchooling`, and `Life_exp`. Countries with higher normalized GDP per capita tend to also have higher normalized mean schooling years and higher normalized life expectancy. Conversely, countries with lower GDP per capita generally show lower mean schooling and life expectancy. For `InfantMortality`, an inverse relationship is typically seen: as GDP, schooling, and life expectancy increase, infant mortality tends to decrease. This suggests that socio-economic development, educational attainment, and healthcare (reflected in infant mortality and life expectancy) are interconnected.

To try adding a different group of 5 countries and compare the results:

Go back to the code cell `BpMap1V1mS1k` and modify the `selected_countries` list with a new set of country names. Then, re-run that cell to see the updated multiline chart and observe new patterns.

Why is normalisation important before plotting these indicators together?

Normalisation is important because the indicators use different units and scales. For example, GDP per capita may have values in the thousands while life expectancy is usually between 50 and 90 years. Without normalisation, variables with larger values would dominate the chart and make comparisons difficult. Normalisation places all variables on a similar scale so patterns can be compared more effectively.

What patterns do you observe between GDP, schooling, and life expectancy?

Countries with higher GDP per capita generally tended to have higher levels of schooling and longer life expectancy. In contrast, countries with lower GDP often showed lower schooling levels and reduced life expectancy. This suggests a positive relationship between economic development, education and overall health outcomes.

What did you observe when using a different group of countries?

When I selected a different group of countries, the overall trend remained similar, although the magnitude of the differences changed. Developed countries generally displayed higher GDP, higher schooling levels and longer life expectancy, while developing countries showed greater variation across these indicators. This demonstrates that economic and educational factors are often associated with health outcomes.